

A 4D Cardiac Deformation Model with Control-Point Gaussian Splatting and Temporal Gaussian Process Priors

Sofia Sannino

Section of Mathematics

Semester Project

Spring 2026

Responsible
Prof. Olivier Verscheure
EPFL / SDSC

Supervisors
David Brüggemann
SDSC

Quentin Duchemin
SDSC



Chapter 1

Introduction

Developing models that are able to learn cardiac deformation patterns from 4D medical images, like cine-MRI, is critical to improve the technological support needed by medical professionals to diagnose heart conditions faster and with increased reliability.

In this work, we define the theoretical framework for a cardiac deformation model that leverages temporal Gaussian process priors to correct control-points prediction on cardiac volumes and make use of Gaussian splatting as rendering technique, to confront predicted medical images and ground-truth.
Furthermore, we propose a geometry-based deterministic algorithm for efficient and accurate control-points extraction from cardiac segmentation masks, to provide valid pseudo ground-truth control points for the pre-training of the deformation model's first stage, a coordinate regression network.
Finally, we show the results of the training and inference experiments on the pre-training of the coordinate regression network, conducted on the ACDC [1] dataset.

1.1 Related work

Gaussian representation in cardiac deformation models. Recent work has explored Gaussian-based representations as an alternative to purely voxel-based or mesh-based cardiac motion models. In particular, Fu et al. proposed Dyna3DGR, a self-supervised framework that combines an explicit 3D Gaussian representation of the cardiac volume with an implicit neural deformation field [2]. In their formulation, the heart anatomy is represented through a set of anisotropic Gaussian primitives, whose positions, covariances and intensities are optimized to reconstruct the cardiac volume, while a deformation network predicts time-dependent transformations of the Gaussian centers and scales. This representation is particularly attractive because Gaussian ellipsoids can influence local image regions and preserve contour information better than discrete voxel representations, allowing the model to bridge representation space and

image space through differentiable volumetric rendering. However, despite its strong reconstruction and tracking performance, Dyna3DGR still presents limitations for physiologically interpretable cardiac deformation modeling. The deformation is driven by learned control nodes and an implicit neural field, whose parameters are not directly tied to anatomically meaningful and consistently identifiable cardiac landmarks. Moreover, in the ACDC setting, the available segmentation supervision is concentrated on the End-Diastole (ED) and End-Systole (ES) phases, and the tracking evaluation is mainly performed at ES. As a result, the deformation is primarily constrained between two clinically relevant states, rather than being explicitly modeled through all intermediate frames of the cardiac cycle. This limits the ability to represent the real continuous temporal evolution of cardiac motion, motivating a frame-wise control-point formulation combined with temporal Gaussian process priors, as proposed in this work.

Gaussian processes in cardiac deformation models. Gaussian processes have recently been introduced in cardiac motion tracking to impose temporal consistency on learned deformation fields. In GPTrack, Yang et al. propose an unsupervised diffeomorphic registration framework in which the Gaussian process (GP) is not used directly in image space, but as a probabilistic prior in the compact latent space of the motion model [13]. More precisely, sequential image features are processed through a bidirectional recurrent architecture, so that information is aggregated both forward and backward across the cardiac sequence. A GP layer is then used to model the latent cardiac motion dynamics, encouraging smooth and consistent temporal trajectories while preserving region-specific variability. This is motivated by the fact that different cardiac structures can move with different amplitudes, phases and directions, while still following a coherent periodic pattern over the heartbeat cycle. The method therefore uses the GP as a regularizing prior over latent motion encodings, improving temporal coherence with respect to pairwise registration approaches that only consider adjacent frames. We took inspiration from their pipeline, although our framework applies GP priors directly to the temporal trajectories of anatomically relevant control points, aiming to obtain an interpretable and frame-wise smooth representation of cardiac deformation.

Gaussian splatting as a rendering technique. Gaussian splatting was introduced by Kerbl et al. as an explicit radiance-field representation based on anisotropic 3D Gaussian primitives coupled with a fast differentiable renderer [7]. Instead of representing a scene through a dense voxel grid or an implicit neural field queried by costly ray marching, the method represents geometry and appearance with localized Gaussian primitives whose position, covariance, opacity and appearance parameters can be optimized directly. This makes Gaussian splatting particularly useful as a rendering technique because it combines the advantages of continuous volumetric representations with the efficiency of explicit point-based rasterization. In the original work, this design enabled high-quality real-time rendering while keeping training times competitive with faster NeRF-based methods, showing that Gaussian primitives can provide both accurate reconstruction and efficient differentiable image formation. For this reason,

Gaussian splatting has become attractive beyond novel-view synthesis, including dynamic and medical settings, where a differentiable rendering stage is useful to compare a predicted geometric representation with image-space ground truth and to propagate the reconstruction error back to the underlying model parameters.

Chapter 2

Methods

2.1 Model overview

Our model aims to build a patient-specific framework for learning and reconstructing a 4D cardiac deformation field, in three-dimensional space and time, from a temporal medical image sequence, combining : a deterministic, geometry-based, control-point prediction algorithm that extracts control points from segmentation masks of each patient's frame, a coordinate regression neural network, temporal Gaussian process (GP) priors on the control point trajectories and, finally, a Gaussian Splatting (GS) stage [7].
We consider, for each patient, the sequence of images that represents a cardiac deformation:

$$\mathcal{I}^p = \{I_t^p\}_{t=0}^T, \tag{2.1}$$

where each I_t^p is a three-dimensional image volume at time t, specific for the patient p. For each frame, we extract control points positioned in relevant spots to capture heart deformation. The objective is to learn a deformation field

$$\Phi^p(\mathbf{x}, t) : \mathbb{R}^3 \times \mathbb{R} \to \mathbb{R}^3 \tag{2.2}$$

that, for each control point at position $\mathbf{x}$ in the cardiac volume, chosen a time step t, gives the new control point's position at time t. We use the word "indirectly" because, with respect to other recent approaches that aimed to directly learn the function Φ [2] - but modeling the deformation just in the two times corresponding respectively to the End-Diastole and End-Systole phases - our approach indirectly builds Φ, leveraging the combination of a coordinate regression network, a temporal GP on the control-points deformation trajectories and a GS module in the rendering stage. A scheme of the model's stages can be found in Fig. 2.1.
In detail, our hybrid pipeline is constructed with the following stages:

- **A sparse dynamic heart structure composed by control points** positioned in relevant spots of the medical 3D volume, obtained through a coordinate regression network. This network is pre-trained using as pseudo ground-truth control

points obtained with a deterministic, geometry-based algorithm that extracts control points coordinates from each frame's segmentation mask. After pre-training, the coordinate regression layer becomes part of the model as a *differentiable* way to extract control points from each patient's frame and that can be further optimized, on top of the optimal pre-training weights, when back propagating from the final loss. Thus, for each patient p and frame I_t^p, let f be the coordinate regression network that maps the 3D volume to the corresponding control points coordinates:

$$\mathbf{C}_t^p = f(I_t^p), \tag{2.3}$$

where $\mathbf{C}_t^p \in \mathbb{R}^{K \times d}$ is the matrix containing the K control points coordinates of dimension d, which in our case is obviously 3.

- **Temporal Gaussian Process (GP) priors** on the control points trajectories that enforce smoothness and uncertainty-aware regularization. In particular, we consider the control points coordinates predicted from f as a noisy version of the real ones, as the network produces output with a certain error. To enforce physiologically plausible smooth motion in this framework, it is a theoretically valid option to model the corrected version of the noisy $\mathbf{C}_t^p$ with a latent GP trajectory that encodes the temporally smoothed and periodic underlying signal, obtaining corrected trajectories that better represent the equally periodic and smoothed cardiac motion:

$$\mathbf{Z}_1^p, ..., \mathbf{Z}_T^p, \tag{2.4}$$

where T is the number of frames for each patient.

- **A dense Gaussian representation** obtained through weighted interpolation of current control points. At each time t, we construct a set of M_t Gaussian splats:

$$G_t = \{g_{m,t}\}_{m=1}^{M_t}, \tag{2.5}$$

where each Gaussian $g_{m,t}$ is parameterized by: $g_{m,t} = (\mu_{m,t}, \Sigma_{m,t}, \alpha_{m,t}, \mathbf{I}_{m,t})$, with mean $\mu_{m,t} \in \mathbb{R}^3$, covariance $\Sigma_{m,t} \in \mathbb{R}^{3\times3}$, weight $\alpha_{m,t} \in \mathbb{R}^+$, intensity descriptor $\mathbf{I}_{m,t}$. Therefore, for each frame, we render the corresponding volumetric prediction using Gaussian splatting [7], with parameters based on control points at time t, and compare the rendered frame with the ground-truth image with a voxel-wise $L1$ loss.

2.2 Dataset and data pre-processing

In order to pretrain the coordinate regression network f, it is necessary to find a way of computing relevant control points in the 3D cardiac volume through a deterministic, geometry-based algorithm. The algorithm we present extracts coordinates from each frame's volume segmentation mask, particularly on tissues borders, to better capture the organ deformation in time, making having available segmentation masks for every frame and for ever patient necessary. Thus, in the following paragraphs, we describe the

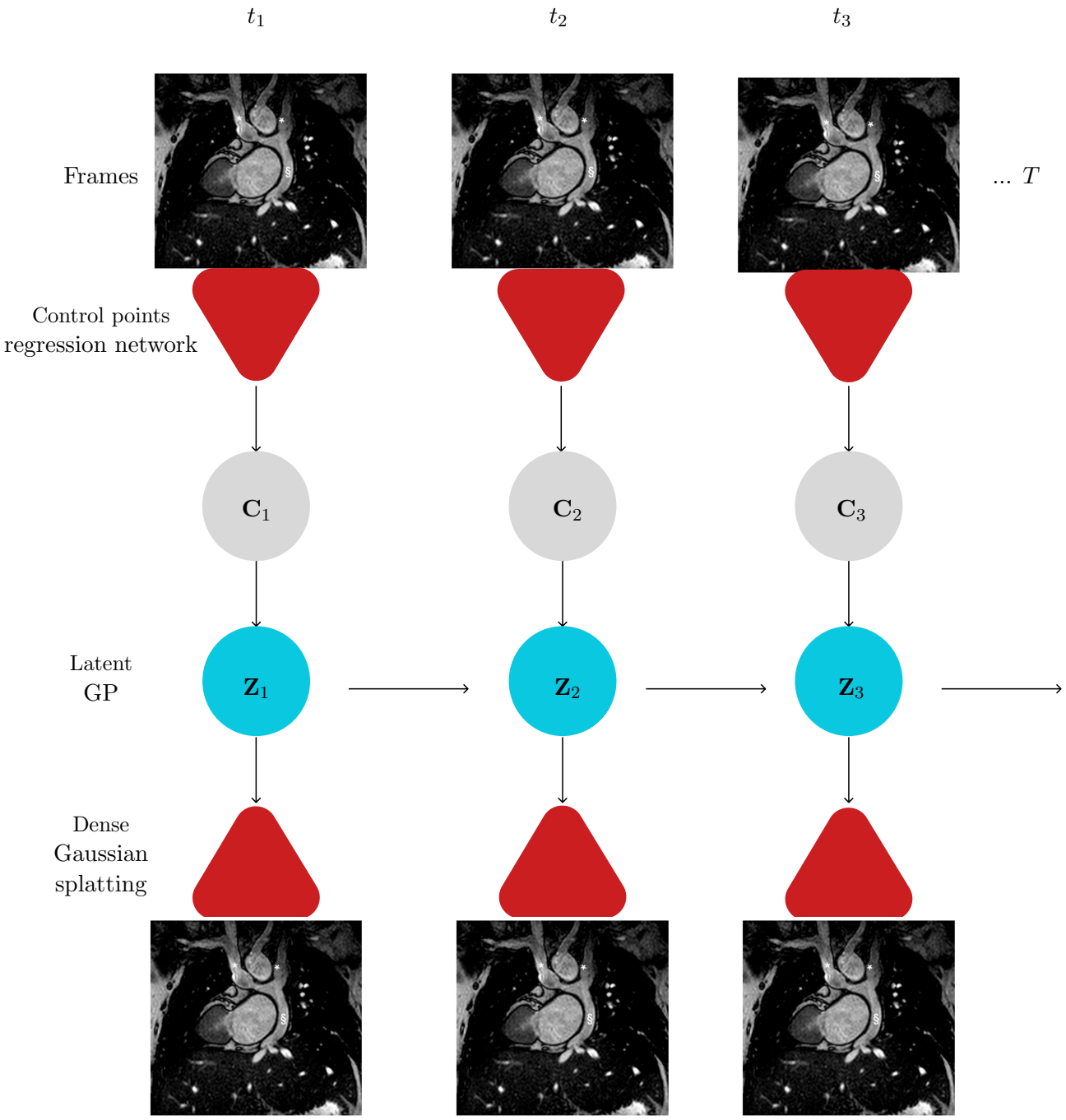


Figure 2.1: Overview of the proposed deformation pipeline, for a patient p, with frames $t = 1, ..., T$.

ACDC dataset [1] that we used in our experiments, how we computed the intermediate

segmentation masks that were not available in ACDC, leveraging a U-Net architecture [12], and how we instead treated the 4D MRI cardiac volume available to obtain a fixed number of frames per patient, a crucial step to build the model pipeline that uses frames at consecutive times as ground-truth.

2.2.1 Dataset

The Automated Cardiac Diagnosis Challenge (ACDC) dataset [1] is a public cardiac MRI benchmark introduced to evaluate automatic methods for cardiac multi-structure segmentation and diagnosis. It consists of short-axis cine-MRI examinations divided into five clinical categories: normal subjects, myocardial infarction with altered left ventricular ejection fraction, dilated cardiomyopathy, hypertrophic cardiomyopathy, and abnormal right ventricle. Each patient is represented by a temporal sequence covering one cardiac cycle, with a patient-specific number of frames, so that the temporal sampling and the observed deformation sequence can vary across cases. For each case, the end-diastolic and end-systolic phases are identified, and expert manual annotations are provided for the left ventricular cavity, right ventricular cavity, and myocardium, making the dataset suitable for both anatomical segmentation and cardiac function analysis. In this project, ACDC is used as a reference 4D cardiac MRI dataset because it provides temporally resolved cardiac images together with clinically meaningful segmentation masks at the two key phases of the cardiac cycle.

2.2.2 Computing intermediate frames

In datasets such as ACDC [1], the 4D cardiac volume is available for each patient, while segmentation masks are provided only for two clinically relevant phases of the cardiac cycle, End-Diastole (ED) and End-Systole (ES). Having a fixed number of frames per patient is crucial for our pipeline, since it allows us to construct standardized 4D volumes on which the deformation field can be learned. At the same time, the availability of frame-wise segmentation masks is required to obtain ground-truth control points for pretraining the network f. However, each 4D volume corresponds to a different patient's cardiac motion and may contain a different number of frames, as well as a different ES positions. This makes it difficult to apply the model pipeline efficiently and consistently across patients. To address this issue, we resampled each 4D volume to contain exactly T frames. Assuming that the ED phase corresponds to the first frame, as in the ACDC convention, let the original sequence contain T_{orig} frames, and let s_{ES} denote the original ES index. The ES frame is mapped to the new temporal grid as

$$s'_{\text{ES}} = 1 + \text{round}\left((T-1)\frac{s_{\text{ES}}-1}{T_{\text{orig}}-1}\right). \tag{2.6}$$

The resampling is then performed separately on the two temporal intervals, from ED to ES and from ES to the end of the cardiac cycle. This ensures that both ED and ES are preserved exactly in the interpolated sequence. For each new frame, we computed its corresponding continuous position m in the original temporal grid. If m is not an

integer, the frame is obtained by linear interpolation between the previous and following original frames. Denoting

$$t_{\text{prev}} = \lfloor m \rfloor, \qquad t_{\text{next}} = \lceil m \rceil, \qquad \alpha = m - t_{\text{prev}},$$

the interpolated frame is defined as

$$I'_t = I_{t_{\text{prev}}} + \alpha \left(I_{t_{\text{next}}} - I_{t_{\text{prev}}} \right). \tag{2.7}$$

If m is an integer, the corresponding original frame is copied directly. Finally, the ED and ES frames are explicitly overwritten with the original frames after interpolation, so that the clinically annotated phases remain unchanged:

$$I'_1 = I_{\text{ED}}, \qquad I'_{s'_{\text{ES}}} = I_{\text{ES}}. \tag{2.8}$$

This procedure provides a temporally standardized 4D representation for each patient while preserving the two reference cardiac phases used by the dataset annotations.

2.2.3 Computing intermediate masks with U-Net

As previously introduced, the control-points, that constitute the ground-truth of the coordinate regression network pretraining, are extracted through a geometry-based pipeline that finds relevant coordinates on tissue borders using the 3D segmentation volume of the heart. Thus, we produced segmentation masks of intermediate frames, training a U-Net architecture on available ED and ES segmentation masks. In particular, the nnU-Net framework was leveraged [5] to perform data pre-processing and training. We used a full-resolution 3D U-Net, since we aimed to capture all the relevant features of 3D heart volumes. We report the details of the implementation used in the following paragraphs.

Data pre-processing and data augmentation. Data pre-processing and data augmentation, during training, included the following steps:

- **Resampling volumes to target spacing**. In particular, in order to take into account the strong anisotropy of the cardiac medical images, we considered the median spacing for the high-resolution axes and the 10th percentile spacing for the low-resolution axis.
- **Normalizing voxel intensities** of the medical images (MRI scans) through Z-Score normalization, in order to standardize the range of intensities across patients and preventing the model to learn unstable patterns.
- **Cropping the image to a target size**, based on its non-zero region, removing empty background areas outside the acquired image content.
- **Data augmentation**, through random rotations, random scaling, intensities augmentations such as Gaussian noise and Gaussian blur, mirroring and flipping.

Loss function. The network is trained using a segmentation loss that combines soft Dice loss and cross-entropy loss. The Dice term directly encourages spatial overlap between the predicted segmentation and the reference annotation, which is especially useful for medical segmentation tasks where foreground structures occupy only a small portion of the image. The cross-entropy term provides voxel-wise supervision and penalizes incorrect class predictions at each spatial location. The final training objective is therefore written as

$$\mathcal{L} = \mathcal{L}_{\text{Dice}} + \mathcal{L}_{\text{CE}}.$$

When deep supervision [9] is enabled, this same loss is also applied to auxiliary decoder outputs at lower resolutions. These auxiliary losses are combined with decreasing weights, so that the highest-resolution prediction remains the main training target while intermediate outputs help improve gradient flow and stabilize training.

Evaluation metrics. Validation performance is evaluated using overlap-based segmentation metrics. The main metric is the Dice score, computed separately for each foreground class and then averaged across all foreground structures to obtain the mean foreground Dice. For a given class, the Dice score is defined as

$$\text{Dice} = \frac{2TP}{2TP + FP + FN},$$

where TP, FP, and FN denote true-positive, false-positive, and false-negative voxels, respectively. This metric measures how well the predicted segmentation overlaps with the ground-truth annotation. For the cardiac segmentation task, Dice scores are reported for each anatomical structure, such as the right ventricle, left ventricle, and myocardium, together with the mean foreground Dice as the main summary measure.

2.3 Control Points extraction algorithm

Pre-train the coordinate regression network in our pipeline is crucial to assure that the weights of this layer are properly initialized to reach convergence in the main training. Thus, we need valid control points coordinates that act as ground-truth in the neural network pretraining phase. To obtain them, we developed a deterministic, geometry-based algorithm that extracts valid control points positions from each frame's segmentation mask, targeting them in strategic zones to better capture the heart deformation, in particular the majority of them is extracted from each different tissue's borders, while some on the internal structure. In the following paragraphs we fully illustrate our method.

2.3.1 Alignment transformation across frames

In the control-points extraction pipeline, a common reference system is first defined by selecting the first segmentation frame of the first patient as the anchor frame. For each patient, the spatial transformation to this reference is computed only once, by registering

the patient's first frame to the anchor frame. The segmentation masks are first binarized and converted into signed distance maps, which provides a smoother representation of the anatomical structures and accounts for anisotropic voxel spacing. An initial Euler 3D transform is then obtained from the mask moments and refined through image registration. The resulting transform is subsequently applied to all frames of the same patient, so that control points are extracted in a consistent anchor coordinate system rather than independently in different patient-specific spaces. This avoids introducing frame-dependent registration variations and ensures that the temporal evolution of the control points mainly reflects cardiac deformation rather than differences in global position, orientation, or image geometry. After extraction, the control points are transformed back to the original reference system of each frame and converted into continuous voxel coordinates, so that they remain directly compatible with the original patient images used by the downstream model.

2.3.2 Uniquely identifying moving control points across frames

To uniquely identifying control points across frames, the pipeline assigns each point an identity through a fixed spherical direction rather than through its absolute coordinates alone, since they are obviously moving because of heart's deformation. In particular, it samples a deterministic set of directions on the unit sphere, including the two poles and a regular grid of intermediate directions defined by the spherical angles θ and ϕ. For each anatomical label, such as left ventricle (LV), right ventricle (RV), and myocardium of the left ventricle (MLV), rays are cast from the center of mass of each label's region in each frame's mask along these fixed directions, aiming at storing the corresponding label surface intersection as the control point corresponding to that direction for that label. Considering that cardiac structures are sufficiently convex, or at least approximately star-shaped with respect to their centroid, each direction identifies one consistent anatomical surface location. Since the same ordered set of directions is used for every frame and every patient, the n-th control point $c_{n,\ell} = (x, y, z)$ of label l always corresponds to the same radial direction (θ, ϕ), such that:

$$(x, y, z) = c_\ell + \rho \begin{pmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{pmatrix}, \qquad \theta \in [0, \pi], \quad \phi \in [0, 2\pi),$$

where c_ℓ is the center of mass of the region of label ℓ and ρ denotes the radial distance from it, making it possible to compare and track control-point motion across time while reducing ambiguity due to changes in pose, shape, or patient anatomy. We fixed the number of directions to N_{dir}, thus obtaining N_{dir} control points corresponding each to a different direction. ρ will be chosen so that the control point is closed to boundary of the respective tissue, to better capture cardiac deformation, as explained in the next paragraph. In summary, each control point is uniquely identified by the label ℓ region in which it is sampled, following the procedure described below, and the spherical direction $(\sin\theta\cos\phi, \sin\theta\sin\phi, \cos\theta)$ with respect to its center of mass c_ℓ.

2.3.3 A geometry-based algorithm to extract control points

To capture relevant control points for each cardiac region (LV, RV, MLV), we developed a geometry-based deterministic algorithm that, given a segmentation mask, extracts for each label region and for each unique direction (θ, ϕ) from its center of mass, coordinates of points near the tissue border, to better capture the organ deformation since tissue boundaries are the zone more affected by repetition of stress and relaxation during a heart beat. The algorithm is structured following the steps reported in 1. The labels $\mathcal{L}$ in our case are the ones corresponding to LV, RV and MLV, the directions are the ones defined in the previous paragraph, and the segmentation masks are the ones available in ACDC dataset alongside the intermediate ones computed using a U-Net, as described above. The intersection with the boundary of each labeled region, along a certain direction, is identified through a Binary Search algorithm described in 2 or a Grid Search algorithm described in 3 as fallback for when the centroid is not inside the tissue region, like in the myocardium case.
After this pipeline, the dataset, obtained through a pre-processing of the ACDC dataset, is composed by : T 3D medical volumes that represent consecutive frames during the cardiac deformation; T corresponding segmentation masks that we used to extract the control points; T sets of corresponding control points, each composed of N_{dir} control points per label (LV, RV, MLV), uniquely identified through a fixed direction $d \in \mathcal{D}$, $|\mathcal{D}| = N_{\text{dir}}$, from each center of mass of the respective label region ℓ, for a total of $K = 3N_{\text{dir}}$ control points per frame.

Algorithm 1 Control point extraction from a segmented structure

Require: Segmented mask S, set of labels $\mathcal{L}$, set of directions $\mathcal{D}$
Ensure: Control points P_ℓ for each label $\ell \in \mathcal{L}$ such that $\forall p_{\ell,d} \in P_\ell$ is uniquely identified by $d \in \mathcal{D}$

```
for each label ℓ ∈ L do
    Extract the binary mask S_ℓ
    Compute the largest connected component of S_ℓ
    Compute its center of mass c_ℓ
    for each direction d ∈ D do
        Cast a ray from c_ℓ along direction d
        Find the intersection with the boundary of S_ℓ, or a point close, through
        Binary Search or Grid Search as fallback
        Store the point p_{ℓ,d}
    end for
end for
```

Algorithm 2 Binary search for boundary control-point extraction

Require: Labeled region S_ℓ, center of mass c_ℓ, direction d, equivalent-sphere diameter diam_ℓ of S_ℓ, maximum number of iterations $N_{\max}$
Ensure: Boundary control point p_{border}

$p_{\text{start}} \leftarrow c_\ell$
$p_{\text{end}} \leftarrow c_\ell + \text{diam}_\ell \cdot d$
if $p_{\text{start}} \notin S_\ell$ **or** $p_{\text{end}} \in S_\ell$ **then**
    Use grid search as fallback
end if
$i \leftarrow 0$
while $i < N_{\max}$ **do**
    $p_{\text{mid}} \leftarrow \frac{p_{\text{start}} + p_{\text{end}}}{2}$
    if $p_{\text{mid}} \in S_\ell$ **then**
        $p_{\text{start}} \leftarrow p_{\text{mid}}$
    else
        $p_{\text{end}} \leftarrow p_{\text{mid}}$
    end if
    $i \leftarrow i + 1$
end while
$p_{\text{border}} \leftarrow p_{\text{start}}$
return p_{border}

Algorithm 3 Grid search for boundary control-point extraction

Require: Labeled region S_ℓ, center of mass c_ℓ, direction d, equivalent-sphere diameter diam_ℓ of S_ℓ, maximum number of iterations $N_{\max}$, δx step size of the grid search towards c_l
Ensure: Boundary control point p_{border}

$p \leftarrow c_\ell + \text{diam}_\ell \cdot d$, $p \notin S_\ell$
$i \leftarrow 0$
while $i < N_{\max} \ \wedge \ p \notin S_\ell$ **do**
    $p \leftarrow p - d \cdot \delta x$
    $i \leftarrow i + 1$
end while
$p_{\text{border}} \leftarrow p$
return p_{border}

2.4 Coordinate regression network

The coordinate regression network is the first learnable component of the proposed deformation pipeline. Its role is to introduce a differentiable mapping from each 3D cardiac frame to a sparse set of control-point coordinates.

Given a patient p and a frame I_t^p, we denote this network by f, so that

$$\mathbf{C}_t^p = f(I_t^p),$$

where $\mathbf{C}_t^p \in \mathbb{R}^{K\times 3}$ contains the predicted coordinates of the K control points at time t. The network is first pretrained in a supervised way using as target the control points extracted from the segmentation masks by the geometry-based algorithm described in the previous section.
After this pretraining stage, f provides an initial sparse representation of the cardiac anatomy directly from the image volume, without requiring the segmentation mask.
In the full model, the predicted control points $\mathbf{C}_t^p$ are then interpreted as noisy observations of the underlying temporally smooth trajectories, which are corrected by the Gaussian process prior before being passed to the Gaussian splatting stage.
In the following paragraphs the architecture of the network is described, which can be summarized as a $3D$ U-Net [12] encoder plus a coordinate head that outputs $\mathbf{C}_t^p \in \mathbb{R}^{K\times 3}$, then confronting it with the ground-truth - the algorithmically obtained control points - with a Huber loss [4]. The architecture of the model is schematically shown in 2.2.

2.4.1 3D U-Net encoder

The coordinate regression network uses the encoder of a 3D full-resolution U-Net [12] as its feature-extraction backbone. The input is the full 3D cardiac volume, and the encoder processes it through a sequence of resolution stages. Each stage is composed of stacked 3D convolutional blocks, where the number of convolutions, feature channels, kernel sizes, and strides are specified stage by stage. Downsampling is performed either through strided convolutions, allowing the network to progressively reduce the spatial resolution while increasing the semantic richness of the extracted features. Each convolutional block can include normalization, non-linearity, and dropout, depending on the chosen configuration. In the original U-Net setting, the encoder can also return intermediate feature maps as skip connections for a decoder; in our coordinate regression network, the encoder instead provides a compact volumetric representation of the input frame, which is then used to regress the K control-point coordinates. This makes the architecture suitable for learning both local anatomical details and broader spatial context from the 3D cardiac image.

2.4.2 Coordinates prediction head

The coordinate prediction head transforms the volumetric features extracted by the encoder into explicit 3D control-point coordinates. It receives as input the bottleneck feature map of the encoder, so the last feature map produced by the encoder stages. If B is the batch size, $C_{\text{bottleneck}}$ is the number of channels in the final encoder stage, and D, H, W are depth, height and width of the medical volume, the input to the head has shape:

$$[B, C_{\text{bottleneck}}, D, H, W]$$

Since this tensor is still spatially structured and potentially high-dimensional, the first operation is an adaptive average pooling layer. This reduces the spatial resolution to a fixed size independent of the original input volume. Thus, given S_{pool} size of the pooling operation, we have:

$$[B, C_{\text{bottleneck}}, S_{\text{pool}}, S_{\text{pool}}, S_{\text{pool}}]$$

This pooling step compresses the encoder output while preserving a global summary of the learned anatomical features. The tensor is then flattened, reaching a dimension of $[B, C_{\text{bottleneck}} * S^3_{\text{pool}}]$. The resulting feature vector is passed through a fully connected multilayer. The first linear layer maps the pooled encoder representation to a hidden coordinate space of dimension H_{hidden} and then two additional fully connected layers with ReLU activations are applied, with the dimension of the features being $[B, H_{\text{hidden}}]$: these intermediate fully connected layers increase the expressive capacity of the coordinate head, since instead of predicting coordinates directly from the pooled encoder features, the network learns a richer nonlinear mapping between the anatomical representation extracted by the encoder and the final control-point positions. Finally, the last linear layer outputs $[B, K \times 3]$ predicting $K \times 3$ scalar values, corresponding to the three spatial coordinates of each of the K control points. A final sigmoid activation function is applied, to reduce the variability of dimension ranges across different patient coordinates, making the regression more stable.

2.4.3 Loss and metrics

During pretraining, the coordinate regression network f is optimized using the Huber loss [4] between the predicted control-point coordinates and the ground-truth coordinates extracted from the segmentation masks. Given the prediction $\hat{\mathbf{C}}^p_t = f(I^p_t)$ and the target control points $\mathbf{C}^p_t \in \mathbb{R}^{K\times 3}$, the loss is computed coordinate-wise and averaged over all control points and spatial dimensions. Given $e = c - \hat{c}$, we have:

$$\mathcal{L}_{\text{Huber}} = \frac{1}{3K} \sum_{k=1}^{K} \sum_{j=1}^{3} \ell_\delta \left(\hat{c}^p_{k,t,j} - c^p_{k,t,j} \right), \tag{2.9}$$

where

$$\ell_\delta(e) = \begin{cases} \dfrac{1}{2}e^2, & \text{if } |e| \le \delta, \\ \delta \left(|e| - \dfrac{1}{2}\delta \right), & \text{if } |e| > \delta. \end{cases} \tag{2.10}$$

The Huber loss was chosen because it combines the stability of a squared error for small coordinate deviations with the robustness of an absolute-error-like penalty for larger deviations. This is useful in our setting because some target control points may be affected by segmentation noise, imperfect boundary extraction, or local anatomical ambiguities. In addition to the training loss, we evaluated the model using the average

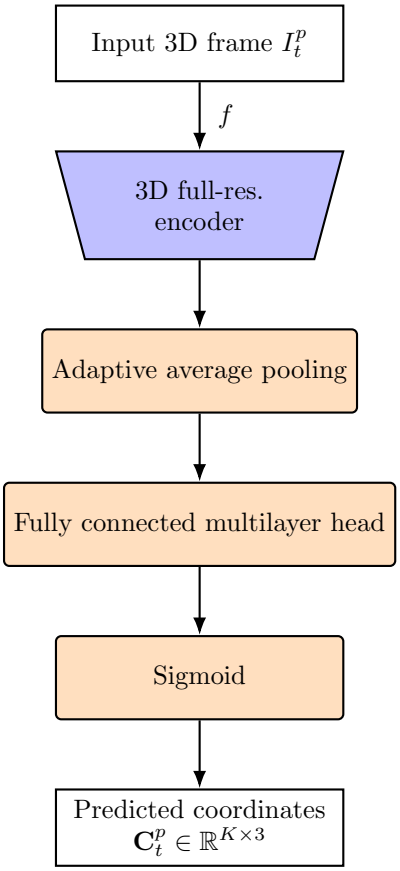


Figure 2.2: Scheme of the coordinate regression network f architecture.

Euclidean distance between predicted and ground-truth control points:

$$d_{\text{avg}} = \frac{1}{K}\sum_{k=1}^{K}\left\|\hat{\mathbf{c}}_{k,t}^{p} - \mathbf{c}_{k,t}^{p}\right\|_2 . \tag{2.11}$$

This metric provides a direct geometric interpretation of the prediction error in coordinate space.

2.4.4 Coordinate normalization

Since the coordinate prediction head uses a final sigmoid activation, its outputs are constrained to the interval $[0, 1]$. For this reason, the ground-truth control-point coordinates are normalized before training. If the k-th control point of a patient p extracted at frame t has voxel coordinates $\mathbf{c}_{k,t}^{p} = (x_{k,t}^{p}, y_{k,t}^{p}, z_{k,t}^{p})$ in an image volume of size (W, H, D), the normalized coordinates are defined as

$$\tilde{\mathbf{c}}_{k,t}^{p} = \left(\frac{x_{k,t}^{p}}{W-1}, \frac{y_{k,t}^{p}}{H-1}, \frac{z_{k,t}^{p}}{D-1}\right). \tag{2.12}$$

The network is therefore trained to predict normalized coordinates $\hat{\tilde{\mathbf{C}}}_t^p \in [0,1]^{K\times 3}$, matching the range imposed by the sigmoid function. At inference time of the pre-training of f, the predicted normalized coordinates are mapped back to voxel space by multiplying each component by the corresponding image dimension. This normalization makes the regression problem numerically more stable and avoids inconsistencies between the coordinate scale and the bounded output range of the network.

2.5 Gaussian Process prior

In this deformation pipeline, the control points coordinates predicted from f are considered as a noisy version of the real ones, since the network produces output with a certain error. Thus, to enforce physiologically plausible smooth and periodic motion, as the deformation of the heart is continuous and of course periodic, it is natural to model the corrected version of the noisy $\mathbf{C}_t^p$ with a latent Gaussian process (GP) trajectory that encodes the temporally smoothed and periodic underlying signal, obtaining the corrected control points trajectories. The GPs are assumed independent across control points and independent across parameter dimensions.
In particular, a per-control-point, per-dimension GP prior $z_p^{(k,r)}(t)$ has been considered, for every patient p, where k indicates the k-th control point at frame t and r one of its dimensions, that in this 3D setting are of course three. Moreover, the conditioned probability $\mathbf{p}(z_p^{(k,r)}(t)|c_p^{(k,r)}(t))$ is obtained considering the GP as a solution of a m-th order linear time invariant stochastic differential equation (LTI SDE) solved numerically with Rauch-Tung-Striebel (RTS) Kalman filtering algorithm. In the following sections, the theoretical framework is described in detail. For a given patient p, we suppress the index p when no ambiguity arises.

2.6 Definition of the GP on each control point coordinate

Let p be a patient whose frames are processed by our deformation pipeline. We suppress the index p since no ambiguity arises.
Let $c_t^{(k,r)} = \mathbf{C}_t[k,r]$ be the r-th coordinate of the k-th control point, obtained as (k,r)-th component of the output of the coordinated regression layer $\mathbf{C}_t = f(I_t)$ at patient's frame I_t at time t, for:

$$k = 1, ..., K \qquad r = 1, ..., d \qquad t = 1, ..., T.$$

K is the total number of control points per frame, d is the control points space dimensionality, which in our case is obviously three, and T is the fixed maximum number of time frames extracted from the cardiac cine-MRI of the patient.
Let $z^{(k,r)}(t)$ denote the r-th coordinate of the k-th control point in the real physiological trajectory. Thus, for every t, we consider $z^{(k,r)}(t)$ as the not-noisy and physical version of $c_t^{(k,r)}$ and we model the temporal frame-wise sequence $z^{(k,r)}(t_1), ..., z^{(k,r)}(t_T)$

as a stochastic process, in particular as a latent Gaussian process of the noisy observations $c_1^{(k,r)}, ..., c_T^{(k,r)}$, for every control point k and every coordinate r. A scheme of this framewrok is shown in Fig. 2.3.

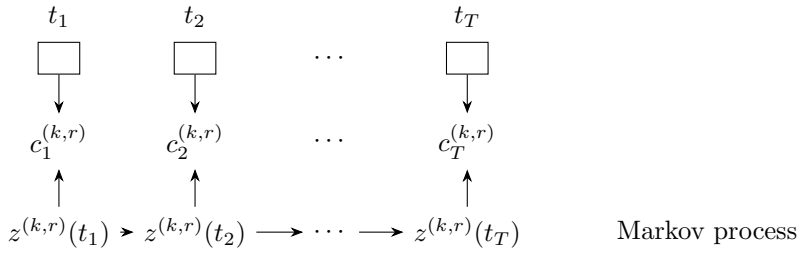


Figure 2.3: Temporal Markov process (GP) for the latent variables and corresponding control-point observations. The model is valid $\forall\,(k,r)$ control point coordinate indipendently.

Thus, given

$$\mathbf{z}^{(k,r)} \in \mathbb{R}^T \qquad , \qquad \mathbf{z}^{(k,r)} = \begin{bmatrix} z^{(k,r)}(t_1) \\ z^{(k,r)}(t_2) \\ \vdots \\ z^{(k,r)}(t_T) \end{bmatrix}, \tag{2.13}$$

we model the latent temporal trajectory with a GP prior,

$$z^{(k,r)}(t) \sim \mathcal{GP}\big(0, \kappa(t,t')\big), \tag{2.14}$$

where κ represents the GP prior kernel, whose properties will be discussed in the following sections, and where the GP mean is chosen to be zero. Therefore, we have

$$\mathbf{z}^{(k,r)} \sim \mathcal{N}(0, K_{TT}), \tag{2.15}$$

where

$$[K_{TT}]_{ij} = \kappa(t_i, t_j). \tag{2.16}$$

The matrix K_{TT} is the covariance matrix of the multivariate Gaussian distribution associated with the temporal trajectory of the r-th coordinate of the k-th control point.

2.7 Closed-form posterior using GP regression

Given the described setting, we now want to compute the latent temporal trajectory $z^{(k,r)}(t_i)$, for $i = 1, \ldots, T$, for each control point k and each component r. A natural probabilistic interpretation is to use GP regression [10]. In particular, we consider the network prediction $c^{(k,r)}(t)$ as a noisy observation of the latent signal:

$$c^{(k,r)}(t) = z^{(k,r)}(t) + \epsilon^{(k,r)}(t), \qquad \epsilon^{(k,r)}(t) \sim \mathcal{N}(0, \sigma^2_{\text{obs}}), \tag{2.17}$$

where σ^2_{obs} is the variance of the observation noise, which accounts for the error introduced by the neural network prediction.

Thus, given

$$\mathbf{c}^{(k,r)} \in \mathbb{R}^T, \qquad \mathbf{c}^{(k,r)} = \begin{bmatrix} c^{(k,r)}(t_1) \\ c^{(k,r)}(t_2) \\ \vdots \\ c^{(k,r)}(t_T) \end{bmatrix}, \tag{2.18}$$

we have that the conditional probability of $\mathbf{c}^{(k,r)}$ given $\mathbf{z}^{(k,r)}$ is

$$p\left(\mathbf{c}^{(k,r)} \mid \mathbf{z}^{(k,r)}\right) = \mathcal{N}\left(\mathbf{c}^{(k,r)}; \mathbf{z}^{(k,r)}, \sigma^2_{\text{obs}}\mathbf{I}_T\right). \tag{2.19}$$

Therefore, by applying the classical result of GP regression [10], we obtain

$$p\left(\mathbf{z}^{(k,r)} \mid \mathbf{c}^{(k,r)}\right) = \mathcal{N}\left(\boldsymbol{\mu}^{(k,r)}, \boldsymbol{\Sigma}^{(k,r)}\right), \tag{2.20}$$

where

$$\boldsymbol{\mu}^{(k,r)} = K_{TT}\left(K_{TT} + \sigma^2_{\text{obs}}\mathbf{I}_T\right)^{-1}\mathbf{c}^{(k,r)}, \tag{2.21}$$

and

$$\boldsymbol{\Sigma}^{(k,r)} = K_{TT} - K_{TT}\left(K_{TT} + \sigma^2_{\text{obs}}\mathbf{I}_T\right)^{-1}K_{TT}. \tag{2.22}$$

Therefore, $\boldsymbol{\mu}^{(k,r)}$ represents the smoothed posterior trajectory given the chosen kernel, which encodes the temporal covariance structure of the deformation. Instead, $\boldsymbol{\Sigma}^{(k,r)}$ represents the remaining uncertainty after smoothing.

The closed-form GP posterior is straightforward, but its computational cost is not negligible. In particular, the computation requires storing the covariance matrix K_{TT}, with space complexity $\mathcal{O}(T^2)$, and inverting $K_{TT} + \sigma^2_{\text{obs}}\mathbf{I}_T$, with time complexity $\mathcal{O}(T^3)$. Hence, this is not the most convenient component when using a large number of control points and a large number of time frames per patient.

2.8 State-space representation of GP in cardiac deformation

It has been shown that there exists a state-space representation of GPs [3]. Hence, one can construct GPs as solutions to m-th order LTI SDEs, leading to an exact inference of latent functions in $\mathcal{O}(n)$ time and memory complexity, where n is the number of observations. These results rely on the use of a particular class of kernel functions, stationary kernels:

$$k(t, t') = k(\tau), \qquad \tau = |t' - t|. \tag{2.23}$$

Therefore, we consider this class in our theoretical framework. For simplicity, we denote

$$z^{(k,r)}(t) = z(t).$$

In particular, we consider the m-th order scalar LTI SDE of the form

$$\frac{d^m z(t)}{dt^m} + a_{m-1}\frac{d^{m-1} z(t)}{dt^{m-1}} + \cdots + a_1 \frac{dz(t)}{dt} + a_0 z(t) = w(t), \tag{2.24}$$

where $a_0, \ldots, a_{m-1}$ are known constants and $w(t)$ is a white-noise process with spectral density $S_w(\omega) = q$. This can be written, using the white-noise notation for SDEs, as a first-order Markov process:

$$\frac{d\mathbf{x}(t)}{dt} = F\mathbf{x}(t) + Lw(t), \tag{2.25}$$

where the state $\mathbf{x}(t)$ contains the derivatives of $z(t)$ up to order $m-1$:

$$\mathbf{x}(t) = \begin{bmatrix} z(t) \\ \frac{dz(t)}{dt} \\ \vdots \\ \frac{d^{m-1} z(t)}{dt^{m-1}} \end{bmatrix}. \tag{2.26}$$

The matrices $F \in \mathbb{R}^{m\times m}$ and $L \in \mathbb{R}^{m\times 1}$ can be written as

$$F = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ -a_0 & -a_1 & -a_2 & \cdots & -a_{m-1} \end{bmatrix}, \qquad L = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}. \tag{2.27}$$

By defining

$$B = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix}, \tag{2.28}$$

we can extract $z(t)$ from $\mathbf{x}(t)$ as

$$z(t) = B\mathbf{x}(t). \tag{2.29}$$

To bridge the state-space representation of GPs and the classical one, we state two important theorems that clarify why it is so important to consider a stationary kernel $k(\tau)$, with $\tau = |t_i - t_j|$. Bochner's theorem states that, for any stationary kernel k, there exists a positive finite measure μ such that

$$k(\tau) = \int e^{i\omega\tau}\, \mu(d\omega). \tag{2.30}$$

If $\mu(d\omega)$ is absolutely continuous with respect to the Lebesgue measure, that is

$$\mu(d\omega) = S(\omega)\, d\omega,$$

then

$$k(\tau) = \mathcal{F}^{-1}(S)(\tau), \qquad S(\omega) = \mathcal{F}(k)(\omega), \tag{2.31}$$

where $\mathcal{F}$ is the Fourier transform.

Moreover, the Wiener–Khinchin theorem states that S is the spectral power density of the GP:

$$S(\omega) = \lim_{T\to\infty} \frac{1}{T}\mathbb{E}\left[|\mathcal{F}_T\{z\}(\omega)|^2\right], \tag{2.32}$$

where $\mathcal{F}_T\{z\}$ is the Fourier transform of z over the time interval T.

As described in previous work [3], the spectral density associated with the state-space model can be written as

$$S(\omega) = B(F + i\omega I)^{-1}LqL^\top \left[(F - i\omega I)^{-1}\right]^\top B^\top. \tag{2.33}$$

At this point, the question on how to obtain F, and q, from S remains. Previous work [3] shows that this is possible for covariance functions whose spectral density can be represented as a rational function of the form

$$S(\omega) = \frac{\text{constant}}{\text{polynomial in } \omega^2}, \tag{2.34}$$

starting from the spectral factorization of $S(\omega)$,

$$S(\omega) = G(i\omega)qG(-i\omega), \tag{2.35}$$

where G is a stable rational transfer function. Therefore, we obtain a way to bridge the two GP representations

$$\begin{aligned} &z^{(k,r)}(t) \sim GP\left(0, k^{(k,r)}(t_i,t_j)\right), \\ &\hat{z}^{(k,r)}(t) = z^{(k,r)}(t) + \epsilon_t^{(k,r)} \end{aligned} \qquad \Longleftrightarrow \qquad \begin{aligned} &\frac{d\mathbf{x}(t)}{dt} = F\mathbf{x}(t) + Lw(t), \\ &z^{(k,r)}(t) = B\mathbf{x}(t) \end{aligned} \tag{2.36}$$

for all (k, r) such that $k = 1, \ldots, K$ and $r = 1, \ldots, d$ control points coordinates, where $k(t_i, t_j) = k(\tau)$ is a stationary, possibly periodic, kernel, and where F and q are obtained from the power spectral density

$$S(\omega) = \mathcal{F}(k)(\omega)$$

of the GP.

2.9 Discretization of the continuous state-space model

Starting from the continuous-time state-space model (2.25), we discretize the process at the observation times

$$t_j, \qquad j = 1, \ldots, T.$$

For each control point k and each parameter dimension r, the resulting discrete-time state-space model is

$$\mathbf{x}_{j+1}^{(k,r)} = A_j \mathbf{x}_j^{(k,r)} + q_j^{(k,r)}, \qquad c_j^{(k,r)} = B\mathbf{x}_j^{(k,r)} + \epsilon_j^{(k,r)}, \tag{2.37}$$

where

$$q_j^{(k,r)} \sim \mathcal{N}(0, Q_j), \qquad \epsilon_j^{(k,r)} \sim \mathcal{N}(0, \sigma_{\text{obs}}^2).$$

Here $\mathbf{x}_j^{(k,r)}$ is the Markovian state associated with the scalar latent trajectory $z^{(k,r)}(t)$, while $c_j^{(k,r)}$ is the observed neural prediction of the corresponding control-point parameter at time t_j. The observation matrix B extracts the first component of the state, so that

$$z_j^{(k,r)} = B\mathbf{x}_j^{(k,r)}. \tag{2.38}$$

Therefore, the neural prediction is interpreted as a noisy observation of the latent GP trajectory, as already stated:

$$c_j^{(k,r)} = z_j^{(k,r)} + \epsilon_j^{(k,r)} = B\mathbf{x}_j^{(k,r)} + \epsilon_j^{(k,r)}. \tag{2.39}$$

The transition matrix is obtained from the continuous-time transition matrix, as also shown in [3] Φ:

$$A_j = \Phi(dt_j), \qquad dt_j = t_{j+1} - t_j, \tag{2.40}$$

where

$$\Phi(\tau) = \exp(F\tau). \tag{2.41}$$

The discrete process noise covariance is

$$Q_j = \int_0^{dt_j} \Phi(dt_j - \tau) L q L^\top \Phi(dt_j - \tau)^\top \, d\tau. \tag{2.42}$$

Since the model is linear and Gaussian, inference can be performed using the Kalman filter followed by the Rauch-Tung-Striebel (RTS) smoothing algorithm [6] [11], where the matrix that model the physics of the process is A_j, the matrix that quantifies the noise of the process not captured by A_j is Q_j, the matrix relating the state to the measurements is B, and the matrix quantifying the error of the measurements is σ_{obs}^2.
The Kalman filter computes the filtering distribution $\forall j = 1, ..., T$:

$$p\left(\mathbf{x}_j^{(k,r)} \mid c_{1:j}^{(k,r)}\right) = \mathcal{N}\left(\mathbf{x}_j^{(k,r)} \mid \mu_j^{(k,r)}, V_j^{(k,r)}\right), \tag{2.43}$$

while, since the whole cardiac sequence is available at inference time, we can then apply the RTS smoother to obtain, $\forall j = T, ..., 1$,

$$p\left(\mathbf{x}_j^{(k,r)} \mid c_{1:T}^{(k,r)}\right) = \mathcal{N}\left(\mathbf{x}_j^{(k,r)} \mid \hat{\mu}_j^{(k,r)}, \hat{V}_j^{(k,r)}\right). \tag{2.44}$$

The RTS smoother it is initialized as

$$\hat{\mu}_T^{(k,r)} = \mu_T^{(k,r)}, \qquad \hat{V}_T^{(k,r)} = V_T^{(k,r)}. \tag{2.45}$$

Finally, the smoothed estimate of the latent scalar trajectory is obtained as

$$z_j^{(k,r)} = z^{(k,r)}(t_j) = B\hat{\mu}_j^{(k,r)}, \qquad \forall (k,r), \qquad j = 1, \ldots, T. \tag{2.46}$$

Using the fact that, given

$$x \sim \mathcal{N}(\mu, \Sigma),$$

one can write

$$x = \mu + \Sigma^{1/2} x_0, \qquad x_0 \sim \mathcal{N}(0, I_d),$$

where x_0 is independent of the parameters, sampling from a Gaussian distribution becomes a differentiable operation. Therefore, we can sample from the smoothed posterior as

$$g_j^{(k,r)} = \hat{\mu}_j^{(k,r)} + \left(\hat{V}_j^{(k,r)}\right)^{1/2} n_j^{(k,r)}, \qquad n_j^{(k,r)} \sim \mathcal{N}(0, I_d), \tag{2.47}$$

and obtain the corresponding sampled trajectory value through

$$z_j^{(k,r)} = B g_j^{(k,r)}. \tag{2.48}$$

This procedure has only μ_0 and V_0 as parameters to be optimized with backpropagation and scales linearly with the number of frames T. In particular, Kalman filtering scales as $\mathcal{O}(Tm^3)$, while RTS smoothing scales as $\mathcal{O}(Tm^2)$. Hence,

$$\mathcal{O}(Tm^3) + \mathcal{O}(Tm^2) \sim \mathcal{O}(T),$$

since m is fixed and usually $\sim \mathcal{O}(1)$.

Chapter 3

Experiments

3.1 U-Net Training

We computed the intermediate frames masks using a 3D U-Net [12]. We used the ED/ED frames and segmentation masks of the ACDC dataset [1] as training samples and then inferred on the intermediate frames to finally compute the masks. In the following paragraphs details of the experiment are reported.

Data pre-processing parameters. All 3D cardiac volumes are resampled to a common voxel spacing of $[5.0, 1.5625, 1.5625]$ mm. This spacing reflects the anisotropic resolution of the data, where the through-plane direction has lower resolution than the in-plane directions. After resampling, the median image size is $[18, 237, 208]$ voxels. During training, patches of size $[20, 256, 224]$ are extracted from the resampled volumes. Image intensities are normalized using z-score normalization, without restricting the normalization statistics to a foreground mask. Image resampling is performed with higher-order interpolation in the in-plane directions and nearest-neighbor interpolation along the low-resolution axis, while segmentation masks are resampled with interpolation with nearest-neighbor. This prevents unnecessary smoothing across the low-resolution axis and preserves the discrete structure of the annotations.

Model parameters. The segmentation model is a full-resolution 3D U-Net. The network has six resolution stages with feature-map widths $[32, 64, 128, 256, 320, 320]$. The first stage uses an anisotropic convolutional kernel of size $[1, 3, 3]$, while the remaining stages use $[3, 3, 3]$ kernels. This choice avoids excessive mixing along the low-resolution through-plane direction at the beginning of the network. The downsampling strides are $[1, 1, 1]$, $[1, 2, 2]$, $[2, 2, 2]$, $[2, 2, 2]$, $[1, 2, 2]$, and $[1, 2, 2]$, so spatial resolution is reduced mainly in the in-plane directions and more conservatively along the depth axis. Each encoder stage contains two convolutional blocks, and each decoder stage also contains two convolutional blocks. The model uses 3D convolutions with bias terms, instance

normalization with affine parameters, ReLU activations, and no dropout. Deep supervision is enabled, so auxiliary outputs from intermediate decoder levels also contribute to the training loss. The batch size is set to 4, and the Dice term is computed per sample rather than over the whole batch.

Optimization parameters. The model is trained with an initial learning rate of 10^{-2} and a weight decay of 3×10^{-5}. The learning rate follows a polynomial decay schedule,

$$\eta(s) = \eta_0 \left(1 - \frac{s}{s_{\max}}\right)^{0.9},$$

where η_0 is the initial learning rate, s is the current training step, and $s_{\max}$ is the maximum number of training steps. Training is performed for 100 epochs, with 250 training iterations and 50 validation iterations per epoch. To reduce the effect of foreground-background imbalance, foreground oversampling is used: 30% of the sampled patches are forced to contain foreground voxels, while the remaining patches are sampled randomly. Deep supervision [9] remains active during training, encouraging useful predictions at multiple decoder resolutions and improving gradient flow through the network.

Results. The results plot of the training are shown in Fig. 3.1. The training loss rapidly decreases, showing that the model is consistently learning. The validation loss reports the same pattern and the gap with respect to train loss is sufficiently small to reassure that the model is not overfitting, showing a small bias and high generalization capability. Moreover, the average Dice score across the foreground classes of validation samples is promising, stably reaching values around 0.9. The epoch duration and the learning rate pattern show a continuous and stable training, with the learning rate signaling a smooth convergence towards the minimum of the loss. Giving these results, we considered the trained weights of the U-Net model capable of producing satisfying inference results if applied to intermediate ACDC frames.

3.2 Control points extraction

Extracting efficiently and correctly control points from segmentation masks with a deterministic algorithm, described in the previous chapter, is an important step to assure quality results in the coordinate regression network pre-training and, therefore, in the entire model pipeline. We present here the results of our control points extraction pipeline.

Control points were extracted deterministically from the segmentation masks for the three foreground cardiac labels, corresponding to labels 1, 2, and 3. For each label, a fixed set of spherical directions was sampled from a grid with $\theta_{\text{num}} = 80$ polar samples and $\phi_{\text{num}} = 20$ azimuthal samples. After removing duplicated directions at the poles, this produced 1562 directions per anatomical structure, for a total of $K = 4686$ control points. The control point along each ray is then estimated iteratively, with a maximum of

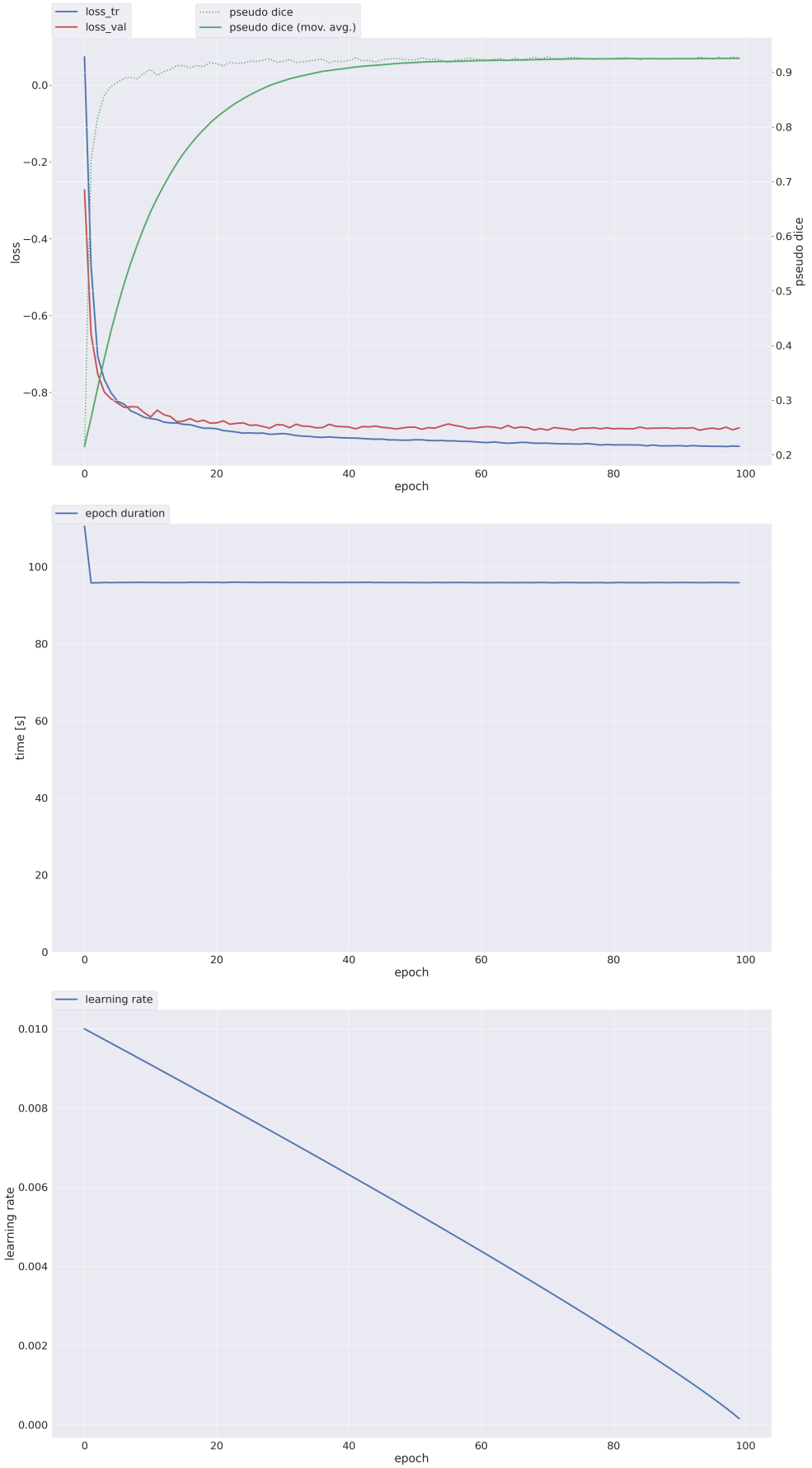
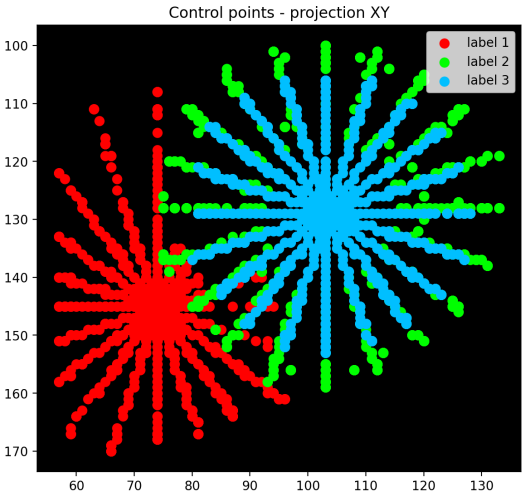
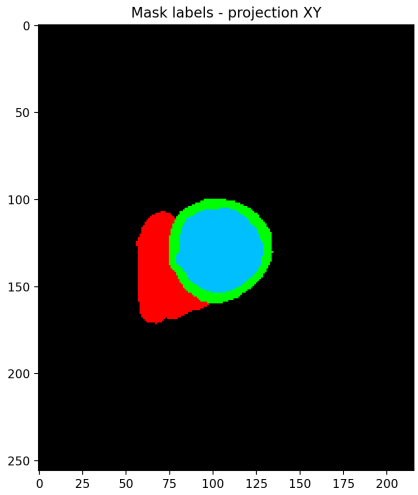


Figure 3.1: Results of training ACDC with U-Net.



(a) Control points sampled, projected in 2D.

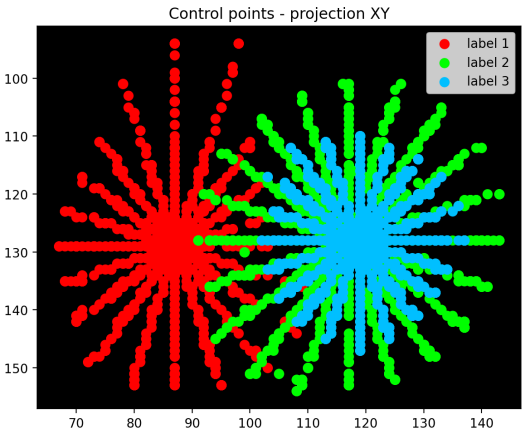


(b) Segmentation mask along height-width plane.

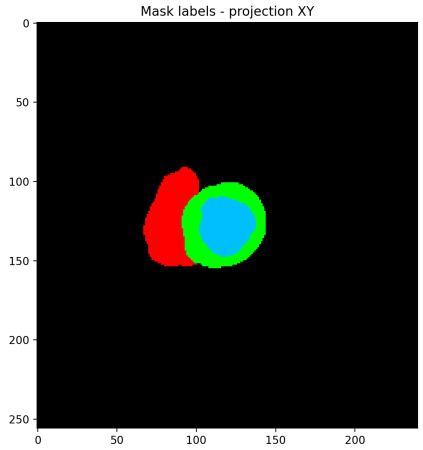
Figure 3.2: Control points extracted projected in 2D vs mask for patient 1.

700 iterations and a step size of 0.5 voxels. If the standard ray search fails, an additional fallback search of 1500 iterations is allowed for robustness. If Binary search fails, Grid Search is applied, with the same number of maximum iterations.

In Fig. 3.2 and Fig. 3.3, the extracted control points are projected onto the width–height plane and compared with the corresponding label masks of the first frame of patient 1 and 21. The projected patterns show that the control points correctly describe the shape of the anatomical regions from which they were sampled, with most points lying on the boundaries of the respective label regions. This behavior is also confirmed by the 3D visualizations in Fig. 3.4 and Fig. 3.5, where the control points extracted from the first-frame masks of patients 1 and 21 are mainly distributed along the tissue boundaries, with additional points located inside the structures, as intended. To further validate the extraction procedure, computed control points were overlayed on each MRI frame for each patient and the more populated height-width plane was plotted. Representative examples are reported in Fig. 3.6. Across all patients and frames, the control-point extraction algorithm described in the previous chapter consistently produces points on the tissue borders, together with a subset of internal points, thus providing valid ground-truth targets for pre-training the coordinate regression network. The time needed to sample 4686 control points on one patient's mask is on average 20 seconds.

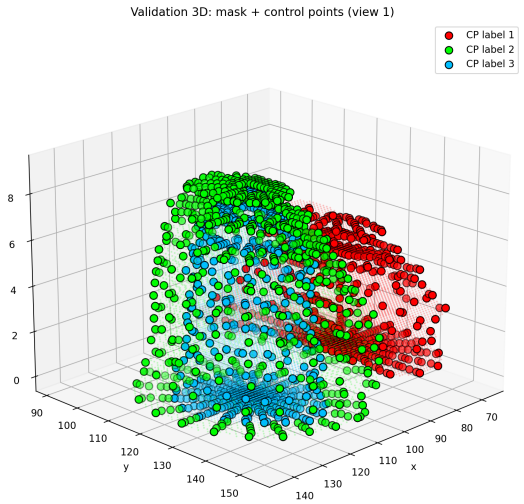


(a) Control points sampled, projected in 2D.

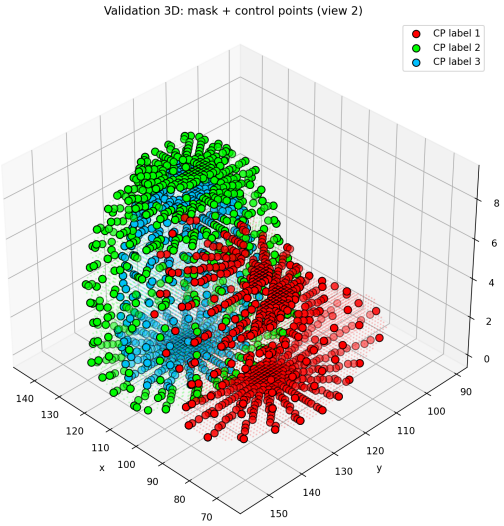


(b) Segmentation mask along height-width plane.

Figure 3.3: Control points extracted projected in 2D vs mask for patient 21.

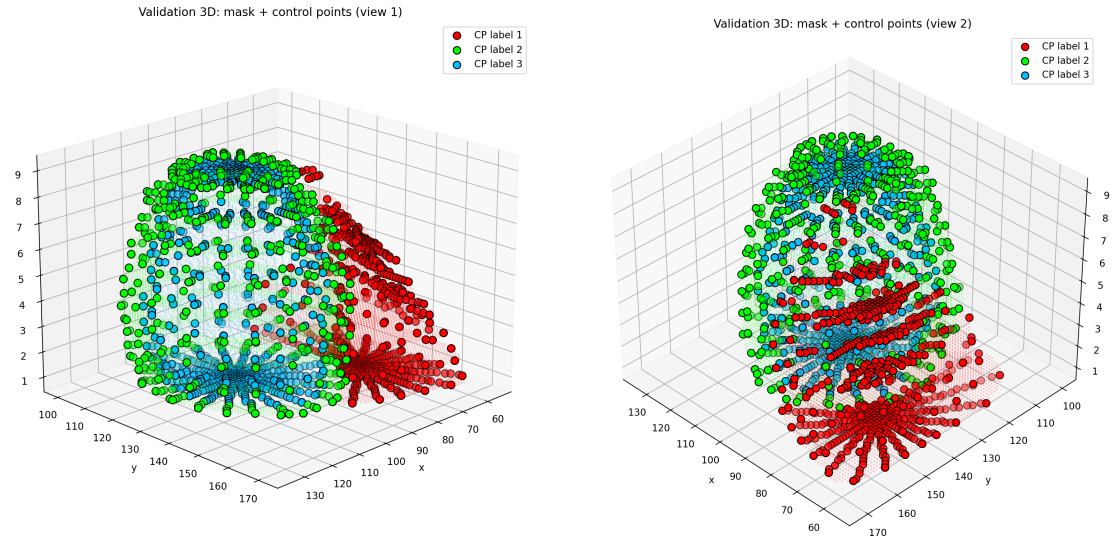


(a) 3D volume view 1.



(b) 3D volume view 2.

Figure 3.5: Control points extracted vs mask in 3D, patient 21.



(a) 3D volume view 1.

(b) 3D volume view 2.

Figure 3.4: Control points extracted vs mask in 3D, patient 1.

3.3 Coordinate regression network pre-training

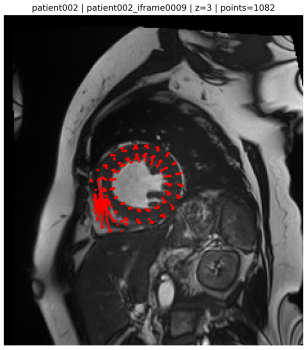
3.3.1 Experiment 1

Data preprocessing parameters. The coordinate regression experiment 1 was carried out on the preprocessed 4D cardiac dataset, using a patient-wise split to avoid mixing frames from the same subject across training and validation. The full dataset contained 60 patients and 1200 frames. 48 patients, corresponding to 960 frames, were used for training, while 12 patients, corresponding to 240 frames, were used for validation. Each input frame was represented as a 3D image volume and normalized using z-score normalization, so that intensity values were centered and scaled before being passed to the network. The target control-point coordinates were also normalized, as described in the previous chapter. In fact, since the network output is constrained with a sigmoid activation, it predicts normalized coordinates in $[0, 1]$, which are later mapped back to voxel coordinates for evaluation and visualization. The training was performed with batch size 1, meaning that each optimization step used one 3D frame and its associated set of control points.

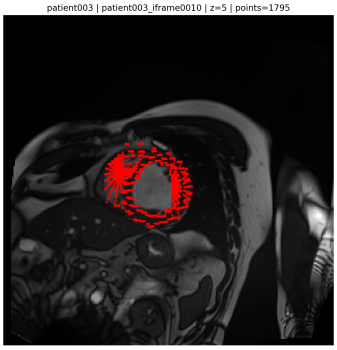
Model parameters. The model is based on a 3D U-Net encoder adapted for coordinate regression. The encoder contains six resolution stages with feature-map widths

$$[32, 64, 128, 256, 320, 320].$$

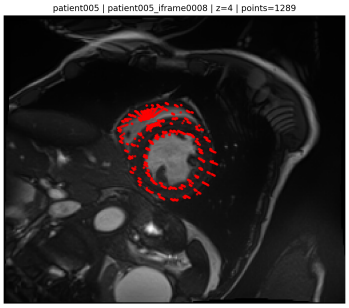
The first stage uses anisotropic convolutional kernels of size $(1, 3, 3)$, while the deeper stages use $(3, 3, 3)$ kernels. This design accounts for the anisotropic nature of the cardiac



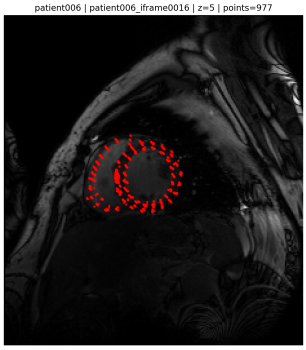
(a) patient 2, frame 9.



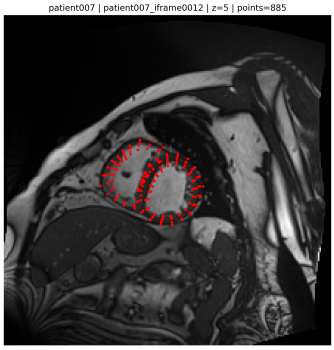
(b) patient 3, frame 10.



(c) patient 5, frame 8.



(d) patient 6, frame 16.



(e) patient 7, frame 12.



(f) patient 21, frame 9.

Figure 3.6: Control points obtained through the deterministic algorithm, projected in 2D on a plane of MRI frames at different times and in different patients.

volumes, limiting early mixing along the through-plane direction. The downsampling strides are

$$(1,1,1),\ (1,2,2),\ (2,2,2),\ (2,2,2),\ (1,2,2),\ (1,2,2),$$

so that spatial resolution is progressively reduced while preserving more information along the depth axis. Each stage contains two convolutional blocks composed of 3D convolution, instance normalization, and ReLU activation. The encoder was initialized from the segmentation checkpoint containing the best weights of U-Net encoder trained previously on ACDC frames to obtain segmentation masks. On top of the encoder, a coordinate regression head applies adaptive average pooling to a fixed $2 \times 2 \times 2$ spatial representation, followed by fully connected layers of width 256. The final linear layer outputs 14058 values, corresponding to 4686 control points with three coordinates each. A sigmoid activation is applied at the output to produce normalized coordinate predictions. The reason to use such big average pooling kernel is to study the behavior of the network, which is been trained on a small amount of samples, and understand if a smaller amount of parameters prevents the model from overfitting.

Optimization parameters. The network was trained as a regression model using the Huber loss [4], between the predicted and ground-truth normalized control-point coordinates, with δ parameter equal to 0.05, since we are dealing with normalized coordinates. Optimization was performed with Adam [8], using an initial learning rate of 10^{-3}, weight decay 3×10^{-5}, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. The learning rate followed a polynomial decay schedule during training,

$$\eta(s) = \eta_0 \left(1 - \frac{s}{s_{\max}}\right)^{0.9},$$

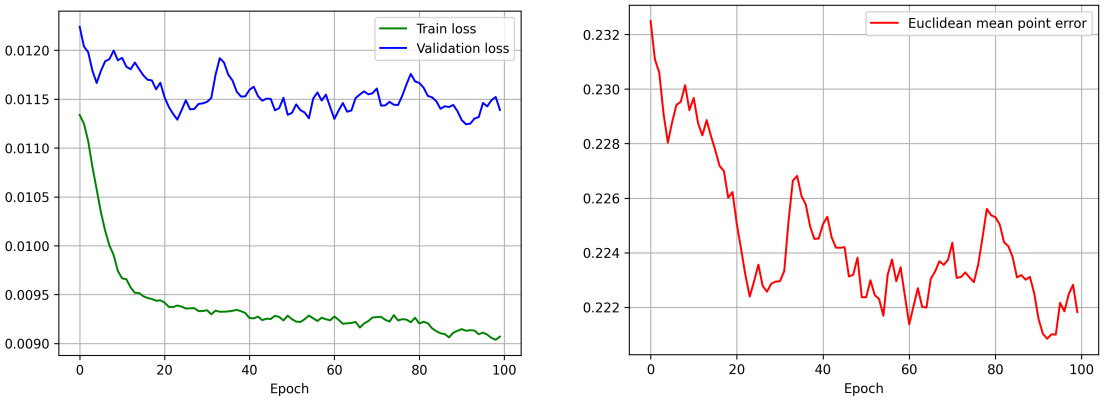
where η_0 is the initial learning rate, s is the current training step, and $s_{\max}$ is the maximum number of training steps. The experiment was run for 100 epochs. At each epoch, the model was evaluated on the validation split to monitor the coordinate regression performance.

Inference parameters. The inference has been carried on 15 patients, for a total of 300 frames, with batch size equal to 1.

Results. As shown in Fig. 3.7, the model is clearly learning: the training loss, validation loss, and mean Euclidean point error all decrease during training. However, a noticeable gap remains between the training and validation losses, suggesting limited generalization to unseen data. This behavior may be due to the relatively small number of training samples, the high variability between patients, or a suboptimal balance between model capacity and regularization. Some inference results on test-patient frames are shown in Fig. 3.8, while the corresponding loss and mean Euclidean point error trends are reported in Fig. 3.9. Qualitatively, the model is able to capture the overall cardiac shape, since the predicted control points often form a plausible cardiac structure. Nevertheless, several failure cases are observed. In particular, the model sometimes fails to recover the correct spatial scale of the cardiac volume, producing control points concentrated in an excessively small region. This is consistent with the inference loss and mean point error curves, which show oscillatory behavior across samples: the model performs well on some frames, but its predictions remain unsatisfactory on others. Overall, these results indicate that the network has learned a meaningful representation of the target geometry, but its robustness across patients and frames is still limited.

3.3.2 Experiment 2

Based on the previous results, the observed scaling issue suggested that the spatial alignment between samples should be improved. Therefore, all frames were padded or cropped to a fixed target shape of $[10, 256, 256]$, reducing variability in input dimensions and encouraging the model to learn coordinates in a more consistent spatial reference frame. In addition, we increased the capacity of the coordinate regression head by reducing the strength of the average pooling operation. Instead of compressing the encoder features too aggressively, the pooling layer was modified to produce a fixed



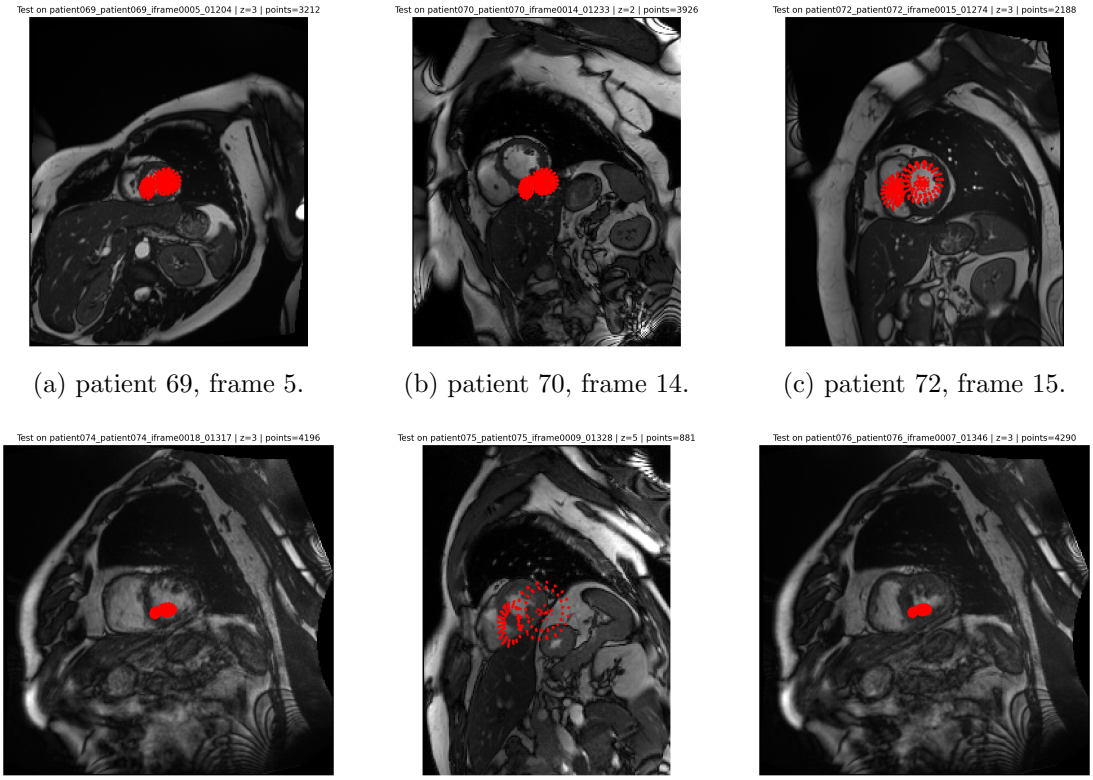
(a) Train and validation loss.

(b) Mean euclidean point error.

Figure 3.7: Training experiment 1.

output tensor of size $8 \times 8 \times 8$, preserving more spatial information before the fully connected layers. The training lasted 100 epochs. All other training settings were kept unchanged with respect to the previous experiment.

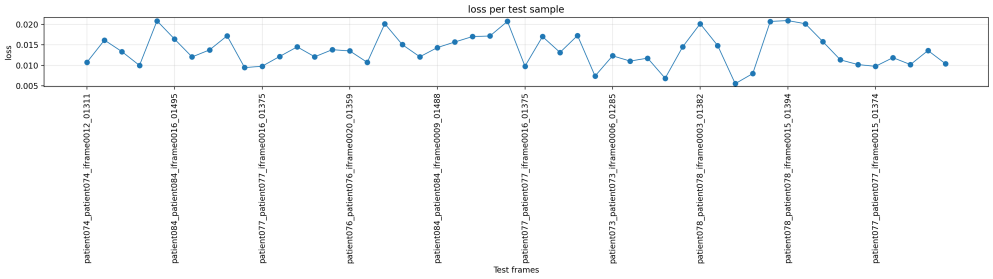
Results. In this new setting, Fig. 3.10 shows that the training loss, validation loss, and mean Euclidean point error all decrease consistently. Although a clear gap between training and validation loss remains, likely due to the limited number of training samples and the variability between patients, the overall training behavior is more stable and exhibits fewer oscillations than in the previous experiment. This suggests that the fixed input size and the less aggressive pooling strategy improve the model's ability to generalize across patients. This improvement is also visible in the qualitative test results shown in Fig. 3.11. The predicted control points define the cardiac shape more accurately, the previous scaling issue is no longer observed, and the points are generally well aligned with the true heart boundaries. The inference loss and mean Euclidean point error are also more stable across test samples, further indicating that this configuration provides better generalization and more reliable coordinate predictions.



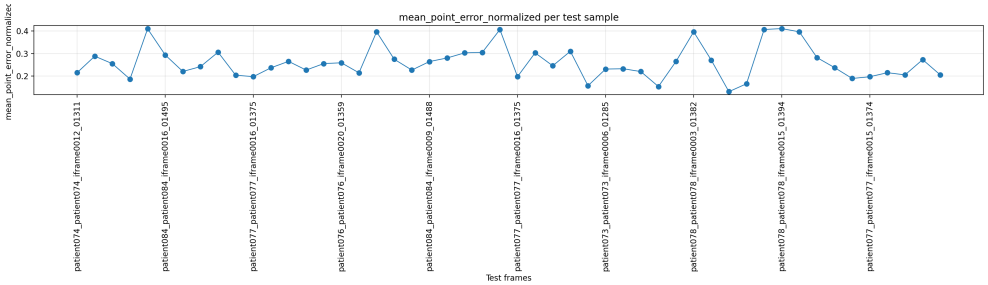
(a) patient 69, frame 5. (b) patient 70, frame 14. (c) patient 72, frame 15.

(d) patient 74, frame 18. (e) patient 75, frame 9. (f) patient 76, frame 7.

Figure 3.8: Inference on some test patients with weights obtained in experiment 1.

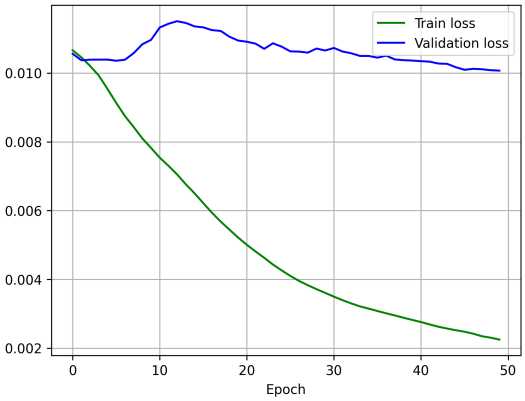


(a) Loss on test samples.

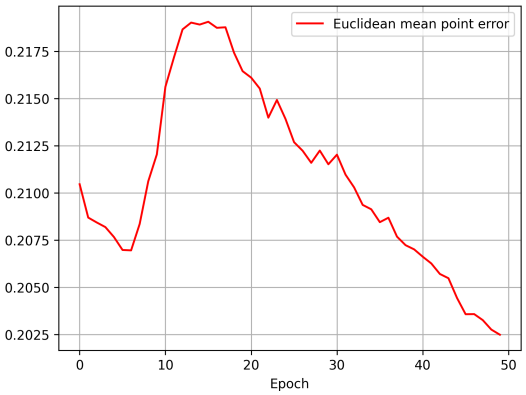


(b) Euclidean point error on test samples.

Figure 3.9: Inference experiment 1.

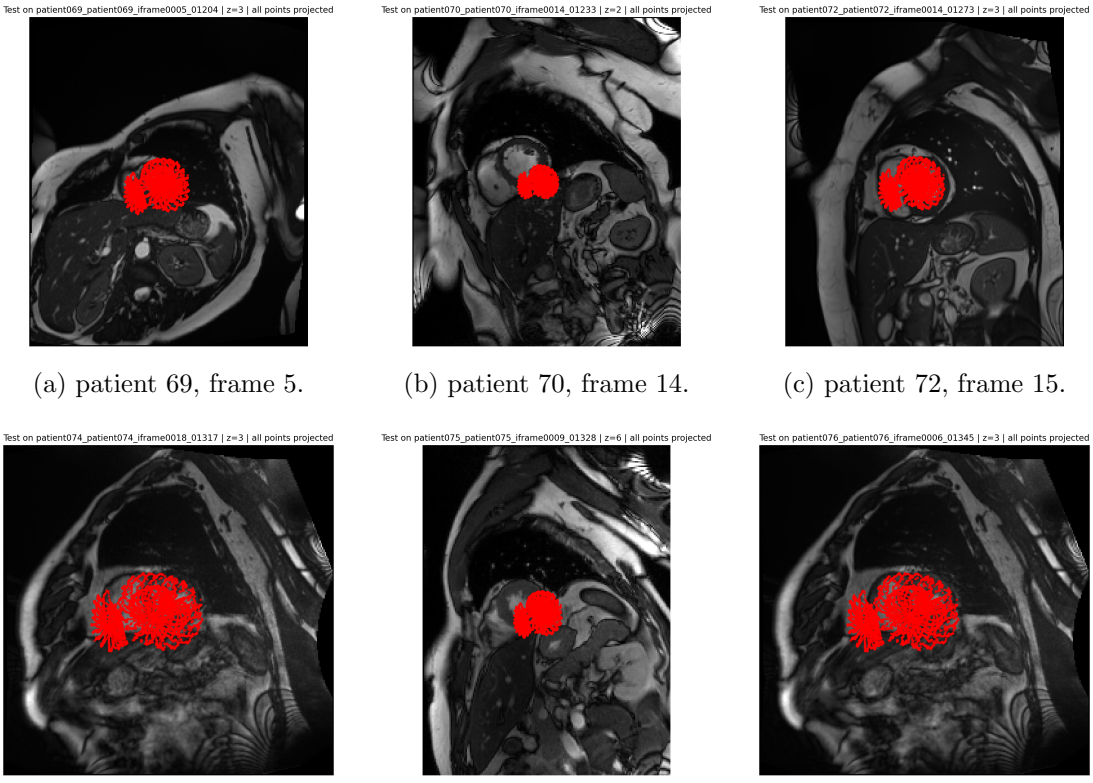


(a) Train and validation loss.



(b) Mean euclidean point error.

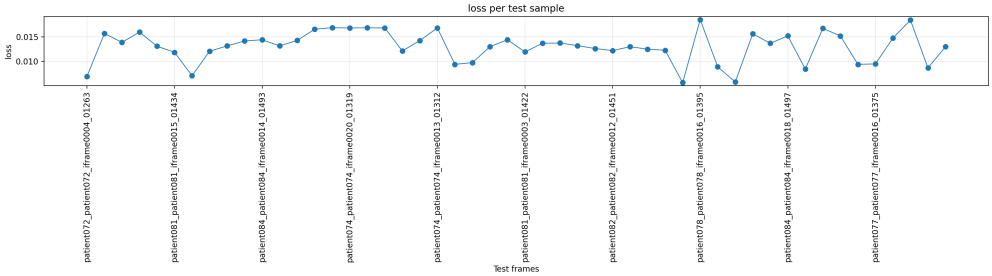
Figure 3.10: Training experiment 2.



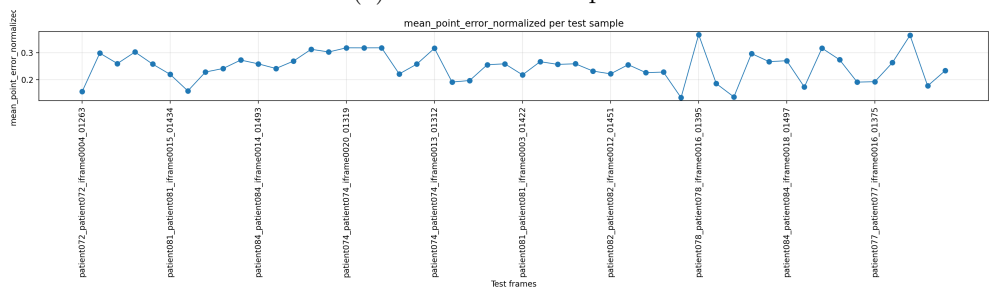
(a) patient 69, frame 5. (b) patient 70, frame 14. (c) patient 72, frame 15.

(d) patient 74, frame 18. (e) patient 75, frame 9. (f) patient 76, frame 7.

Figure 3.11: Inference on some test patients with weights obtained in experiment 2.



(a) Loss on test samples.



(b) Euclidean point error on test samples.

Figure 3.12: Inference experiment 2.

Chapter 4

Conclusions

In this work we defined the theoretical framework of a cardiac deformation model that leverages a simple U-Net encoder and a latent Gaussian process to capture and reproduce cardiac motion. We showed how to represent the Gaussian process as solution of a linear time invariant stochastic differential equation and how to take advantage of it to optimize the computational cost needed to find the latent smooth trajectory from the noisy observation obtained through the coordinate regression network, making it linear in the number of time frames per patient.
We effectively developed a deterministic geometry-based algorithm to extract relevant control points from cardiac segmentation masks, especially at tissue borders and some in their internal part, to efficiently capture cardiac deformation pattern, in a small amount of time.
Finally, we trained and tested a coordinate regression network that make use of a 3D U-Net encoder and a head composed with an average pooling layer and a fully connected multilayer to efficiently predict reasonable estimates of the control points on patient's MRI 3D volumes, constituting a differentiable way to extract control points that can be corrected through the Gaussian process layer and, after performing Gaussian splatting, optimized through back propagation.

The control points extraction pipeline effectively sample relevant control points across the cardiac volume, especially on the borders, by balancing Binary Search, when possible, and Grid Search, as well as using extra iterations to converge to a point when needed. Although fallback methods and extra iterations were not used often, it would be interesting to further increase the efficiency of the algorithm by developing a better balance between efficient sampling and fallback methods, as well as to implement new algorithms for control points search that reach K valid control points per frame in less than 20 seconds on average.

The implemented coordinate regression network, with a simple U-Net encoder and an

average pooling plus MLP head, is able to reconstruct relevant control points pattern on cardiac volumes, although not always with a perfect accuracy. To improve the performance, further experiments with an hyperparameter tuning of the average pooling output dimension can be done, to better assess where lies the best trade-off between a sufficient model capacity and a regularization that avoids overfitting in a dataset with a small amount of samples. Moreover, training performing data augmentation on the frames, such as mirroring, flipping, intensity augmentations, random rotations, coherently transforming ground-truth control points coordinates, can be an option to improve the generalization capabilities of the model. Overall, with the current results, the pretrained network can be introduced in the general pipeline giving a good enough disposition of control points that can be smoothed and corrected by the Gaussian process layer and backpropagation, as suggested by the test results obtained on unseen samples.

The general results obtained provide the theoretical and technical basis to continue the implementation of this cardiac deformation model, that defines an interpretable and computationally feasible framework to predict cardiac motion, continuing the research towards models that help medical professionals improve patients life, with technologies that can make cardiac illnesses diagnosis faster and more reliable.

Bibliography

[1] Olivier Bernard, Alain Lalande, Clement Zotti, Frederick Cervenansky, and al. Deep learning techniques for automatic MRI cardiac multi-structures segmentation and diagnosis: Is the problem solved? *IEEE Transactions on Medical Imaging*, 37(11):2514–2525, November 2018.

[2] Xueming Fu, Pei Wu, Yingtai Li, Xin Luo, Zihang Jiang, Junhao Mei, Jian Lu, Gao-Jun Teng, and S. Kevin Zhou. Dyna3DGR: 4d cardiac motion tracking with dynamic 3d gaussian representation. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*, volume LNCS 15961, pages 164–174. Springer Nature Switzerland, September 2025.

[3] Jouni Hartikainen and Simo Särkkä. Kalman filtering and smoothing solutions to temporal gaussian process regression models. In *2010 IEEE International Workshop on Machine Learning for Signal Processing*, pages 379–384. IEEE, 2010.

[4] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[5] Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2):203–211, 2021.

[6] R. E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME, Journal of Basic Engineering*, 82(1):35–45, 1960.

[7] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4):139:1–139:14, 2023.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[9] Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, and Zhuowen Tu. Deeply-supervised nets. In *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*, volume 38 of *Proceedings of Machine Learning Research*, pages 562–570. PMLR, 2015.

[10] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.

[11] H. E. Rauch, F. Tung, and C. T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965.

[12] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241. Springer, 2015.

[13] Jiewen Yang, Yiqun Lin, Bin Pu, and Xiaomeng Li. Bidirectional recurrence for cardiac motion tracking with gaussian process latent coding. In *Advances in Neural Information Processing Systems*, volume 37, 2024.